

Veri Güvenliği: Aktarımda, Beklemede, Kullanımda

CEN429 Güvenli Programlama — Hafta 3

Dr. Öğr. Üyesi Uğur CORUH · 02.10.2026

0. Temel kavramlar (sıfırdan)

Neden bu bölüm?

Bu hafta "şifreleme", "nonce", "anahtar türetme" gibi terimler geçecek.

Hiçbirini bilmediğinizi varsayıyoruz.

Önce hepsini **tek tek** tanımlayalım.

Verinin üç hâli

- **Aktarımda:** ağda giderken (TLS).
- **Beklemede:** diskte dururken (dosya şifreleme).
- **Kullanımda:** bellekte işlenirken (en zor).

Her hâl farklı koruma ister.

Şifreleme nedir?

- **Şifreleme:** okunur veriyi (açık metin) bir **anahtarla** okunamaz hale (şifreli metin) getirmek.
- **Çözme:** anahtarla geri açmak.
- Güvenlik **anahtarın** gizliliğine bağlı.

Simetrik vs asimetrik

- **Simetrik:** tek anahtar (AES). Hızlı.
- **Asimetrik:** açık + gizli çift (RSA/ECC). Anahtar dağıtımı kolay.
- Pratikte **birlikte** (hibrit).

Özet (hash)

- **Özet:** veriden hesaplanan sabit parmak izi (SHA-256).
- Tek yönlü; veri değişirse özet değişir.
- Bütünlük ve imzanın temeli.

MAC ve imza

- **MAC:** simetrik; mesaj değişmedi + doğru taraftan.
- **İmza:** asimetrik; kim imzaladı (inkâr edilemezlik).
- İkisi de **bütünlük** sağlar.

AEAD

- **AEAD:** gizlilik + bütünlük **birlikte** (AES-GCM).
- Ayrı MAC uğraşmadan ikisini verir.
- Modern tercih.

IV / nonce / tuz

- **IV/nonce:** her şifrelemede **benzersiz** başlangıç değeri.
- **Tuz (salt):** paroladan anahtar türetirken eklenen rastgele değer.
- Üçü de **gizli değil**, ama **tekrarsız/benzersiz** olmalı.

Rastgelelik ve CSPRNG

- **CSPRNG**: kriptografik olarak güvenli rastgele üreteç (işletim sisteminin `getrandom` / `BCryptGenRandom`).
- `rand()` **güvensizdir**.
- Anahtar/nonce/tuz bundan üretilir.

Anahtar türetme (KDF)

- **KDF:** bir sırdan (parola ya da ana anahtar) **anahtar üretme**.
- **PBKDF2/Argon2:** paroladan (yavaş, tuzlu).
- **HKDF:** ana sırdan çok anahtar.

Anahtar hiyerarşisi

- Tek anahtar her işte kullanılmaz.
- Ana anahtar → türev anahtarlar (veri, oturum).
- **Kripto-periyot:** her anahtarın ömrü.

İleri gizlilik

- **İleri gizlilik:** her oturum için yeni anahtar.
- Uzun vadeli anahtar sızsa bile **eski** oturumlar çözülemez.
- Modern TLS sağlar.

TLS (kısaca)

- **TLS:** ağda güvenli iletişim protokolü (HTTPS'in altında).
- Anahtar anlaşması + AEAD + sertifika doğrulama.
- viii. bölümde göreceğiz.

Şimdi hazırız

Terimler:

verinin üç hâli · şifreleme · simetrik/asimetrik · özet · MAC/imza · AEAD · IV/nonce/tuz · CSPRNG · KDF/HKDF · anahtar hiyerarşisi · ileri gizlilik · TLS

Şimdi: verinin üç hâli ve güvenlik kabuğu.

Bugünün planı (3 saat)

Saat	Konu
1	Veri durumları · Şifreleme temelleri Demo 1 · Rastgele sayılar · Kripto API'leri · IV/nonce Demo 2–3
2	Parola ve HKDF Demo 4–5 · Anahtar yönetimi · TLS 1.3 Demo 6 · TLS'i kodda kurmak, fail-open
3	Beklemede Demo 7 · Maskeleye · Kullanımda Demo 8 · Kabuklar Demo 9 · Whitebox · Proje

Öğrenme çıktıları: ÖÇ.2 (şifreleme, güvenli iletişim) · ÖÇ.4 (güvenli kanal)

Kısa tarihçe — veriyi korumanın araçları

- **1976** Diffie–Hellman (açık anahtar) · **1977 RSA** ve **DES**
- **2001** — **AES** (Rijndael) DES'in yerini alır
- **2007** — **GCM** standart olur: **AEAD** çağı (gizlilik + bütünlük birlikte)
- **1994** → **2018** — SSL → TLS 1.0 → **TLS 1.3**

Kural buradan çıkar: kendi kriptonu yazma, **AEAD** kullan, **anahtarı** doğru yönet.

Demolar nasıl çalışıyor?

- Tek kaynak, iki platform: **Windows (Visual Studio 2022)** ve **WSL/Linux (GCC)**
- Kripto ortak başlıktan: **Linux → OpenSSL**, **Windows → BCrypt (CNG)** — OpenSSL kurmaya gerek yok
- Derle: Windows `.\build.ps1` · WSL `./build.sh`
- Çalıştır: Windows `.\demo.ps1` · WSL `sh demo.sh`
- İkili dosyalar her demonun `bin\windows` / `bin/linux` klasöründe

 **Etik:** ağ yalnız `localhost` ; sertifikalar bizim; teknikleri yalnız kendi bilgisayarınızda deneyin.

1. Verinin üç hâli ve güvenlik kabuğu

Neden veriye odaklanıyoruz?

- İlk iki hafta "hata → saldırı → düzeltme" ile **programın kendisini** koruduk.
- Bu hafta odak **veriye** kayıyor: parola, ödeme anahtarı, kişisel bilgi.
- Bir sır, yaşam döngüsü boyunca **üç ayrı hâlde** bulunur.
- Her hâlin tehdidi de savunması da **farklıdır**.

Bir sırrın üç hâli

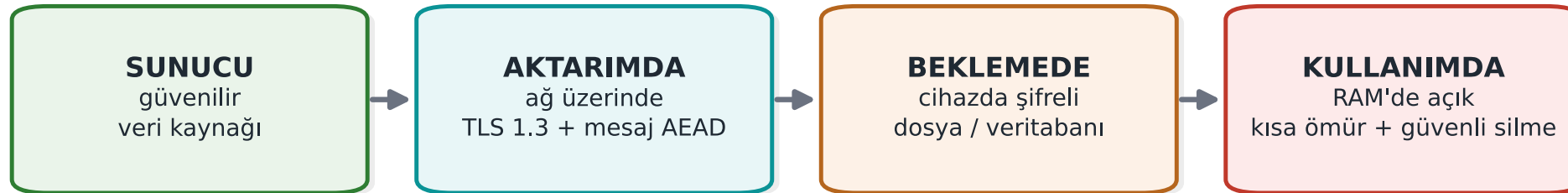
Hâl	Nerede	Tehdit	Savunma
Aktarımda	Ağ	Dinleme, MITM	TLS + pinning + mesaj AEAD
Beklemede	Disk/DB	Dosya çalınması	Alan şifreleme (AEAD)
Kullanımda	RAM	Bellek dökümü	Kısa ömür, güvenli silme

Her hâlin tehdidi de savunması da **farklı**. Hepsini birden düşünmek gerekir.

Üç hâl — tek bakışta

Verinin üç hâli

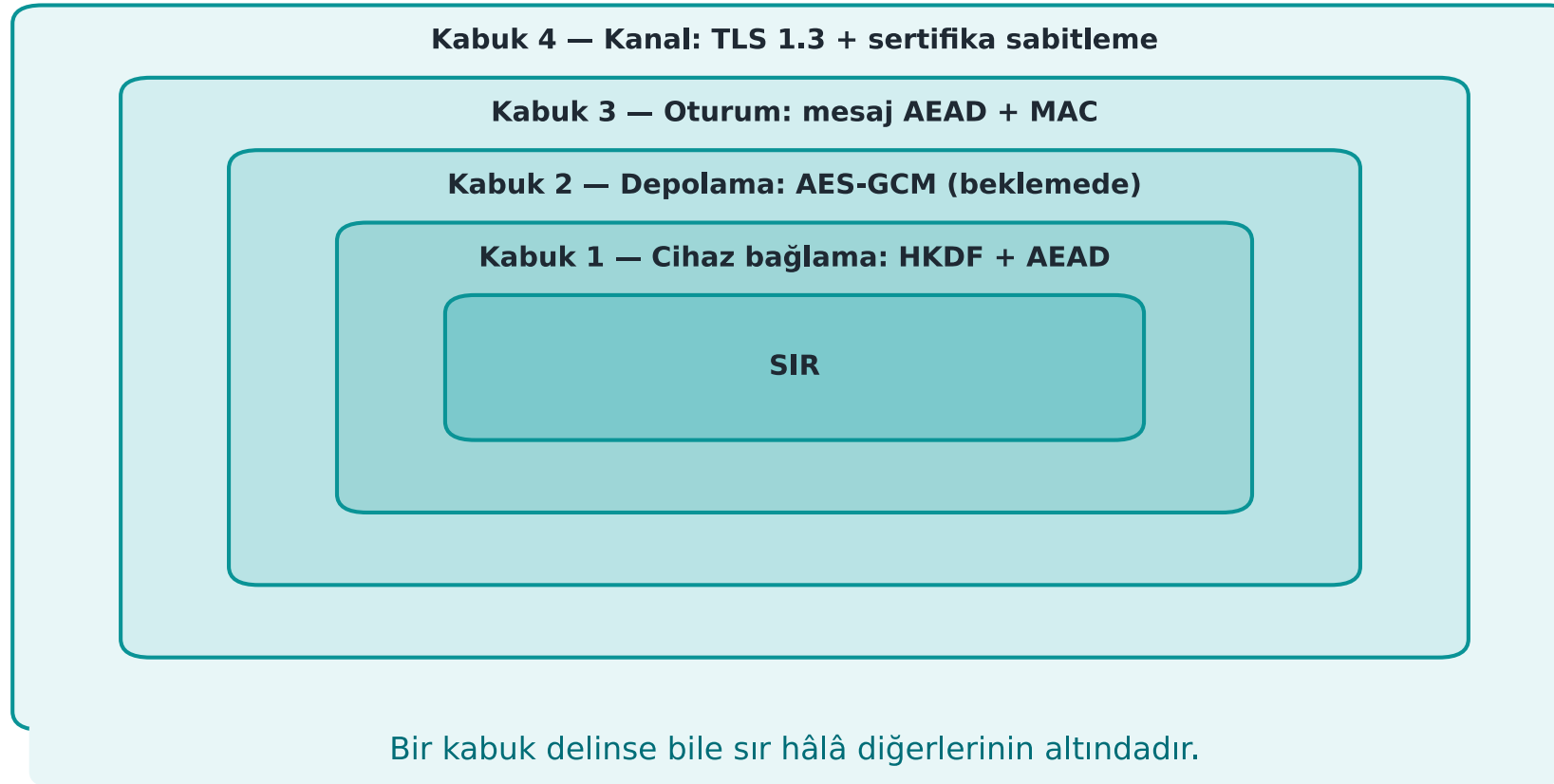
sunucudan çıkıp kullanıcının GÜVENİLMEZ cihazına gider



Her hâlin tehdidi ve savunması farklıdır; üçü birden korunmazsa zincir kopar.

Güvenlik kabukları: derinlemesine savunma

sırta ulaşmak için dört kabuğu da kırmak gerekir



Örnek mimari: telefon ↔ sunucu

- **Telefon:** güvenilmez ortam — kullanıcı (yani olası saldırgan) cihazın sahibi; root/hata ayıklayıcı olabilir.
- **Sunucu:** güvenilir taraf.
- Her arayüzde sorulur: hangi veri, hangi hâlde, kimlik nasıl doğrulanıyor, gizlilik mi bütünlük mü?

2. Şifreleme temelleri

Kriptografinin dört ailesi — şema

Kriptografinin dört ailesi: hangi araç neyi korur?

tek bir "şifreleme" yoktur; hedefe göre araç seçilir

Simetrik şifreleme

GİZLİLİK

AES-GCM, ChaCha20

Özet (hash)

PARMAK İZİ

SHA-256 — anahtar yok

MAC

BÜTÜNLÜK + KİMLİK

HMAC — paylaşılan anahtar

Dijital imza

BÜTÜNLÜK + İNKÂR EDİLEMEZLİK

Ed25519 — özel anahtar

En sık hata: gizlilik ile bütünlüğü karıştırmak. Şifreleme tek başına bütünlük vermez.

Kriptografi tek şey değildir

- Farklı hedefler için **farklı araçlar** vardır.
- En sık karıştırılan dört aile: şifreleme, özet, MAC, imza.
- Hangisi **gizlilik**, hangisi **bütünlük**, hangisi **ikisi birden** verir?

Hangi araç neyi korur?

Araç	Sağlar	Örnek
Simetrik şifreleme	Gizlilik	AES, ChaCha20
Asimetrik şifreleme	Gizlilik + anahtar dağıtımı	RSA, EC
Özet (hash)	Parmak izi (anahtarsız)	SHA-256
MAC	Bütünlük + kimlik (paylaşılan anahtar)	HMAC
İmza	+ inkâr edilemezlik	Ed25519
AEAD	Gizlilik + bütünlük birlikte	AES-GCM

✗ Yalnız şifrelemek bütünlük vermez · ✗ Kendi algoritmanı yazma (Kerckhoffs)

En sık yapılan iki hata

1. **Yalnız şifrelemek, bütünlüğü unutmak.** Şifreli metin (ör. AES-CTR) saldırganın **fark edilmeden** bit değiştirmesini engellemez.
2. **Kendi şifreni/kipini yazmak.** Kerckhoffs ilkesi: güvenlik algoritmanın değil **anahtarın** gizliliğine dayanır.

Standart, denenmiş yapıları kullanın; ev yapımı kript o neredeyse her zaman kırılır.

Simetrik neden tek başına yetmez?

- **Simetrik (AES):** çok hızlı.
- Ama iki tarafın **aynı gizli anahtarı** bilmesi gerekir.
- Bu anahtarı güvensiz bir ağdan **nasıl** paylaşırsınız?

Cevap bir sonraki slaytta.

Asimetrik neyi çözer, neye mal olur?

- **Asimetrik (RSA/EC):** herkesin bir **açık** (dağıtılır) ve bir **özel** (gizli) anahtarı var.
- Anahtar dağıtımı sorununu çözer.
- Ama işlemler **yavaştır**; büyük veriye uygun değil (100–1000× yavaş).

Hibrit: ikisi birlikte

- **Hibrit:** asimetrik ile küçük bir **simetrik anahtar** taşınır; veri onunla AEAD'le şifrelenir.
- Gerçek sistemler hep böyle çalışır.

TLS tam olarak budur: ECDHE ile oturum anahtarı → trafik AES-GCM ile.

Simetrik vs Asimetrik — karşılaştırma

Özellik	Simetrik (AES)	Asimetrik (RSA/EC)
Hız	Çok hızlı	Yavaş (100–1000×)
Anahtar	Tek, paylaşılan gizli	Açık/özel çift
Kullanım	Büyük veri şifreleme	Anahtar taşıma, imza
Bu haftada	Dosya/DB/oturum	TLS anahtar kurulumu (H10)

Blok şifre nedir?

- **Blok şifre:** veriyi sabit boyutlu **bloklar** halinde şifreleyen algoritma.
- **AES:** tek seferde **16 baytlık** bir bloğu şifreler.
- Bloktan **uzun** veri için bir **kip** (mode of operation) gerekir.

Blok mü, akış mı? Kip önemli

AES bir **blok** şifresidir (16 bayt). Uzun veri için **kip** gerekir:

Kip	Durum
ECB	✗ desen sızdırır (Demo 3)
CBC	rastgele IV ister, tek başına bütünlük yok
CTR	akışa çevirir, nonce tekrarına çok hassas (Demo 2)
GCM	✓ CTR + MAC = AEAD

ChaCha20: akış şifre

- **Akış şifresi:** blok kavramı yok; veriyle aynı uzunlukta bir **anahtar akışı** üretip XOR'lar.
- ChaCha20 zaten akış şifresidir.
- + **Poly1305 MAC** = AEAD (ChaCha20-Poly1305, RFC 8439).

Sonuç aynı: **gizlilik + bütünlük tek pakette.**

Özet (hash): tek yönlü parmak izi

- **Anahtarsızdır.** Yalnız verinin **değişip değişmediğini** gösterir.
- Saldırgan veriyi değiştirip **özetini de yeniden hesaplayabilir.**
- Bu yüzden özet tek başına bütünlük **kanıtı değildir.**
- Eski MD5/SHA-1 çakışmaya açık; **SHA-256/SHA-3** kullanın.

MAC: paylaşılan anahtarla bütünlük

- **MAC** (ör. HMAC-SHA-256): **paylaşılan gizli anahtar** ister.
- "Bu mesajı, anahtarı bilen biri gönderdi ve değişmedi" der.
- İki taraf da anahtarı bildiği için MAC **inkâr edilemezlik sağlamaz**.
- MAC karşılaştırması **sabit zamanlı** olmalı (`CRYPTO_memcmp`), düz `memcmp` değil.

İmza: inkâr edilemezlik

- **Dijital imza** (Ed25519, RSA-PSS): **açık/özel** anahtar çiftiyle çalışır.
- Yalnız özel anahtar sahibi imza üretebilir; herkes açık anahtarla doğrular.
- Bu yüzden **inkâr edilemezlik** sağlar.
- Sertifikalar ve sürüm imzalama (Hafta 10) bununla çalışır.

Özet / MAC / İmza: hangisi ne zaman?

Soru	Araç
"Dosya bozuldu mu?" (anahtarsız)	Özet (SHA-256)
"Anahtarı bilen biri mi yolladı, değişti mi?"	MAC (HMAC)
"Kesinlikle şu kişi imzaladı, inkâr edemez"	İmza (Ed25519)
"Hem gizle hem bütünlüğü kanıtla"	AEAD (AES-GCM)

⚠ Özet tek başına bütünlük **kanıtı değildir** (saldırgan özeti de yeniler).

AEAD çoğu zaman doğru cevaptır

- Ayrı ayrı "şifrele + MAC ekle" kurmak yerine **AEAD** kullanın.
- İki işi **tek çağrıda**, doğru sırada (encrypt-then-MAC) ve daha az hatayla yapar.
- Ayrı MAC yalnız şifrelenmeyecek ama bütünlüğü gereken veriler için (ya da AEAD'nin AAD alanıyla) gerekir.

AEAD: tek çağrıda gizlilik + bütünlük

Demo 1 nedir?

- Demo 1 · `code/week-03/01-aes-gcm-dosya` — AEAD, NIST SP 800-38D.
- Bir dosyayı AES-256-GCM ile şifreler.
- Çıktı biçimi: `[12 bayt nonce][şifreli metin][16 bayt etiket]`.
- Sonra şifreli metnin ve yalnız etiketin bir baytını bozup çözmenin **reddedildiğini** görüyoruz.

Demo 1 — kod

```
kripto_rastgele(nonce, 12);           // her mesaja YENI nonce
kripto_gcm_sifrele(anahtar, nonce, 12, NULL, 0,
                  duz, duz_boy, cik+12, etiket);
int ok = kripto_gcm_coz(...);         // etiket dogrulanir
if (!ok) { /* REDDET */ }
```

Demo 1 — gerçek çıktı

```
ADIM 3  Cozuldu ve DOGRULANDI (53 bayt)
ADIM 4  1 bayt degistir -> DOGRULAMA BASARISIZ, REDDEDILDI
ADIM 5  yalnız etiketi degistir -> yine REDDEDILDI
```

Demo 1 — ADIM 2 ve 3: nonce, doğru çözme

- **ADIM 2:** Rastgele 12 baytlık bir nonce üretildi, dosyanın başına yazıldı.
- Nonce **gizli değildir** (dosyada açık durur); yalnız **benzersiz** olması gerekir.
- **ADIM 3:** Aynı anahtar ve nonce ile çözüldü, etiket tuttu, düz metin geri geldi.

Tuz, IV, nonce — şema

Tuz, IV ve nonce: üçü de gizli değil, üçü de farklı iş yapar

kariştirmek en sık yapılan kriptu hatasıdır

TUZ (salt)

paroladan anahtar türetmede

kural: her kayıt için RASTGELE

bozulursa: önhesaplı tablolar işe yarar

IV

CBC gibi kiplerde

kural: öngörülemez olmalı

bozulursa: aynı düz metin aynı görünür

NONCE

GCM gibi AEAD kiplerinde

kural: anahtar başına BİR KEZ

bozulursa: gizlilik VE bütünlük çöker

Üçü de açıkça saklanabilir; güvenlik gizliliklerinden değil, TEKRARLANMAMALARINDAN gelir.

Demo 1 — ADIM 4 ve 5: kurcalama → red

- **ADIM 4:** Şifreli metnin 20. baytı XOR ile değiştirildi. GCM etiketi bütün şifreli metin üzerinde hesaplandığı için **tek bayt bile** etiketi bozar → çözme reddedildi.
- **ADIM 5:** Bu kez yalnız **etikete** dokunuldu; yine reddedildi.
- Etiket, veriyle **ve** anahtarla birlikte hesaplanır; saldırgan onu anahtarsız üretemez.

Demo 1 — kural

- ✓ Bir şey şifreliyorsan **AEAD** kullan (AES-GCM / ChaCha20-Poly1305)
- ✓ Her mesaja **benzersiz nonce**
- ✓ Etiket **her zaman doğrula**; başarısızsa **hiçbir çıktı üretme** (fail-closed)
- ✓ Bütünlüğü ayrı adıma bırakma

Demo 1 — sık hatalar kontrol listesi

- [] Nonce her mesajda benzersiz mi? (Sabit nonce = felaket)
- [] Etiket saklanıyor ve çözerken doğrulanıyor mu?
- [] Doğrulama başarısızsa çıktı üretmeden mi çıkılıyor?
- [] Anahtar koda gömülü değil, güvenli bir kaynaktan mı geliyor?
- [] AES-CBC/CTR tek başına (MAC'siz) kullanılmıyor, değil mi?

3. Rastgele sayılar ve kriptoloji API'leri

Üç tür rastgelelik — şema

Üç tür "rastgele": hangisi nerede kullanılır?

anahtar tahmin edilebilirse algoritmanın gücü anlamsızdır

Sözde rastgele (PRNG)

`rand()`, `Math.random()` — hızlı, tahmin edilebilir; ASLA güvenlikte

Kriptografik (CSPRNG)

`getrandom()`, `BCryptGenRandom` — anahtar, IV, nonce, tuz için

Gerçek rastgelelik (TRNG)

donanım gürültüsü — işletim sisteminin havuzunu besler

Kural: güvenlikle ilgili her değer CSPRNG'den gelir; kendi üreticinizi yazmayın.

Neden rastgelelik bu kadar önemli?

- Anahtar, IV, nonce, tuz, oturum kimliği — hepsi tek bir varsayıma dayanır: **tahmin edilemezlik**.
- Dünyanın en iyi şifreleme algoritması, anahtarı tahmin edilebilir bir üreteçle oluşturulmuşsa **hiçbir şey** korumaz.
- Bu bölümde önce **yanlış** yollara, sonra **doğru** yola bakacağız.

Üç tür "rastgele"

Tür	Örnek	Tahmin edilebilir mi?	Kullanım
İstatistiksel	<code>rand()</code> , <code>mt19937</code> , Java <code>Random</code>	Evet	Oyun, simülasyon
Kriptografik (CSPRNG)	<code>getrandom</code> , <code>BCryptGenRandom</code> , <code>RAND_bytes</code>	Hayır	Anahtar, IV, nonce, tuz, belirteç
Gerçek (entropi)	Donanım gürültüsü	Hayır	CSPRNG'yi tohumlamak

Kural: güvenlikle ilgili her rastgele değer **CSPRNG**'den

`mt19937` 'nin 624 çıktısını gören, sonraki bütün çıktıları hesaplar

Hata 1 — zamanla tohumlamak

```
srand(time(NULL));  
unsigned char anahtar[16];  
for (int i = 0; i < 16; i++)  
    anahtar[i] = rand() & 0xFF;
```

Saldırgan anahtarın hangi gün üretildiğini biliyorsa, olası tohum sayısı bir günde yalnız **86.400**. Bir dizüstü bilgisayar hepsini saniyeler içinde dener.

Hata 2 — entropiyi yanlışlıkla silmek

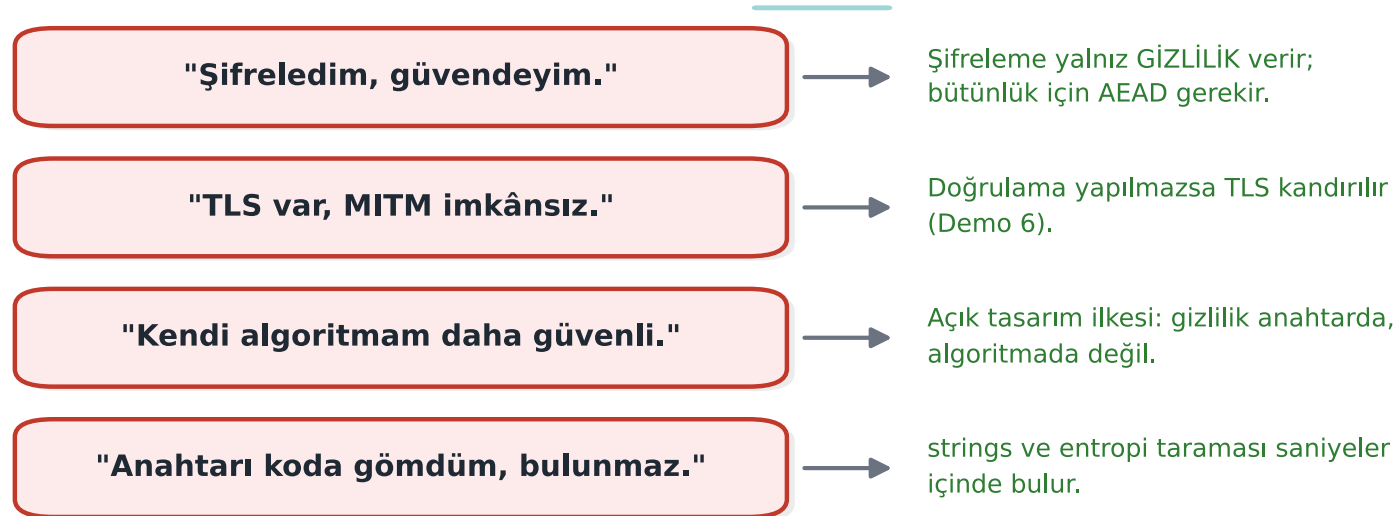
Debian OpenSSL felaketi (2008): bir paketleyici, bir uyarıyı susturmak için OpenSSL'in rastgele sayı havuzuna entropi ekleyen iki satırı kaldırdı.

- İki yıl boyunca tek entropi kaynağı **süreç kimliği** (≤ 32.768 değer) oldu.
- Saldırganlar bütün olası anahtarları önceden üretip listeledi (**CVE-2008-0166**).

Dört yaygın yanlış anlama — şema

Dört yaygın yanlış anlama

her biri sahada gerçek bir açığa dönüşmüştür



Dördünün ortak kökü aynı: güvenliği bir ÜRÜN sanmak, oysa o bir KARAR zinciridir.

Hata 2 — çıkarım

- Kriptografik koda **anlamadan dokunmayın**.
- Rastgele üretcin çıktısını **istatistiksel olarak test etmek**, tahmin edilemezliğini **kanıtlamaz**.

Hata 3 — aralığa indirgerken sapma

$r \% 6$ masum görünür. r 0–255 tekdüze ise:

$$256 = 42 * 6 + 4$$

- 0–3 değerleri **43'er** kez, 4–5 değerleri **42'şer** kez çıkar.
- Zar için önemsiz; parola/OTP üretirken tahmin edilebilirliği artırır.
- Doğru yöntem: **reddetme** (rejection sampling) — sonraki slaytta.

Doğru yol: işletim sisteminin üretici

```
/* Linux (3.17+ / glibc 2.25+) */
while (n > 0) {
    ssize_t r = getrandom(p, n, 0);
    if (r < 0) { if (errno == EINTR) continue; return -1; }
    p += r; n -= (size_t)r;
}

/* Windows */
BCryptGenRandom(NULL, buf, n, BCRYPT_USE_SYSTEM_PREFERRED_RNG);

/* OpenSSL */
if (RAND_bytes(buf, n) != 1) { /* ASLA devam etme */ }
```

⚠ Dönüş değerini denetlemezsen "rastgele" sandığın **sıfır** anahtarla çalışırsın

Dönüş değerini neden denetlemeli?

- Rastgele üretic **başarısız olabilir**.
- Denetlenmezse tampon **ilklendirilmemiş** (ya da sıfır) kalır.
- Program "rastgele" sandığı **sabit** bir anahtarla çalışmaya devam eder.
- Hata durumunda tek doğru davranış: **işlemi durdurmak**.

Reddetme yöntemiyle sapmasız aralık

```
uint32_t aralikta_rastgele(uint32_t ust) {          /* Tarif 11.11 */  
    uint32_t sinir = (uint32_t)(-ust) % ust, r;    /* 2^32 mod ust */  
    do rastgele_bayt(&r, sizeof r); while (r < sinir);  
    return r % ust;  
}
```

Fikir: 2^{32} değerini, `ust` 'e **tam bölünen** en büyük kısmını kullan, kalanı at.

Rastgele değerler — kullanım listesi

Değer	Gizli?	Benzersiz?
Anahtar	Evet	Evet
GCM nonce (12 B)	Hayır	Asla tekrar (rastgele: $\leq 2^{32}$ mesaj/anahtar)
CBC IV	Hayır	Tahmin edilemez
Tuz · Oturum belirteci	Hayır · Evet	Evet

AEAD: dört girdi, iki çıktı

AEAD (AES-256-GCM): girdiler ve çıktılar

tek işlemde gizlilik + bütünlük



Nonce aynı anahtarla İKİ KEZ kullanılırsa gizlilik ve bütünlük birlikte çöker.

AAD: şifrelenmez ama değiştirilirse etiket tutmaz — dosya başlığı, sürüm, kayıt kimliği

1. haftadaki "Kasa": türetme parametreleri AAD → yinleme sayısı değiştirilemez

Şimdi kriptoloji API'sinin içini açıyoruz

- Demo 1'de işi hazır yardımcı fonksiyonlara bıraktık.
- Şimdi OpenSSL'in **EVP** arayüzünü satır satır yazıyoruz.
- Amaç: kütüphaneyi kullanırken nerede hata yapılabileceğini görmek.
- Kural hâlâ geçerli: **kendi algoritmanı yazmazsın**, ama hazır algoritmayı **doğru çağırmak** da bir beceridir.

EVP şifreleme — Adım 1: algoritmayı seç

```
EVP_EncryptInit_ex(ctx, EVP_aes_256_gcm(), NULL, NULL, NULL);
```

- Algoritmayı seç (**AES-256-GCM**).
- Anahtarı ve nonce'u **henüz verme** — sıradaki adımda.

EVP şifreleme — Adım 2: nonce boyu

```
EVP_CIPHER_CTX_ctrl(ctx, EVP_CTRL_GCM_SET_IVLEN, 12, NULL);
```

- Nonce (IV) uzunluğunu ayarla.
- GCM için önerilen **12 bayt**; varsayılan da budur ama açık yazmak iyi bir alışkanlıktır.

EVP şifreleme — Adım 3: anahtar + nonce

```
EVP_EncryptInit_ex(ctx, NULL, NULL, anahtar, nonce);
```

- Anahtarı ve nonce'u **şimdi** ver.
- İlk `Init` çağrısında `NULL` bırakılan yerler burada doldu.

EVP şifreleme — Adım 4: AAD

```
EVP_EncryptUpdate(ctx, NULL, &n, aad, aad_n);
```

- AAD'yi ver: çıktı tamponu **NULL**.
- AAD **şifrelenmez**, yalnız etikete katılır (dosya başlığı, sürüm, kayıt kimliği).

EVP şifreleme — Adım 5: veriyi şifrele

```
EVP_EncryptUpdate(ctx, sifreli, &n, acik, acik_n);
```

- Açık metni şifrele.
- Büyük veride bu adım **parça parça** tekrarlanabilir.

EVP şifreleme — Adım 6: bitir (ZORUNLU)

```
EVP_EncryptFinal_ex(ctx, sifreli + n, &n);
```

- GCM akış kipinde çalıştığı için **dolgu yoktur**; bu çağrı genelde 0 bayt yazar.
- Ama çağrılması **zorunludur**: etiket burada hesaplanır.

EVP şifreleme — Adım 7: etiketi al

```
EVP_CIPHER_CTX_ctrl(ctx, EVP_CTRL_GCM_GET_TAG, 16, etiket);
```

- Etiket al ve şifreli metinle birlikte sakla.

Yedi adım — bir arada

```
EVP_EncryptInit_ex(ctx, EVP_aes_256_gcm(), NULL, NULL, NULL); /* 1 algoritma */
EVP_CIPHER_CTX_ctrl(ctx, EVP_CTRL_GCM_SET_IVLEN, 12, NULL); /* 2 nonce boyu */
EVP_EncryptInit_ex(ctx, NULL, NULL, anahtar, nonce); /* 3 anahtar+nonce */
EVP_EncryptUpdate(ctx, NULL, &n, aad, aad_n); /* 4 AAD (çıktı yok) */
EVP_EncryptUpdate(ctx, sifreli, &n, acik, acik_n); /* 5 veri */
EVP_EncryptFinal_ex(ctx, sifreli + n, &n); /* 6 bitir: ZORUNLU */
EVP_CIPHER_CTX_ctrl(ctx, EVP_CTRL_GCM_GET_TAG, 16, etiket); /* 7 etiket */
```

Her çağrının dönüşü denetlenir · tek çıkış noktası (goto son) · EVP_CIPHER_CTX_free

Çözme — adım 1-2: aç, etiketi ayarla

```
EVP_DecryptUpdate(ctx, acik, &n, sifreli, sifreli_n); /* açık metni YAZAR... */  
EVP_CIPHER_CTX_ctrl(ctx, EVP_CTRL_GCM_SET_TAG, 16, etiket); /* Final'dan ÖNCE */
```

- `DecryptUpdate` açık metni **hemen** tampona yazar — henüz doğrulanmadı!
- Beklenen etiket `Final` 'dan **önce** verilir.

Çözme — adım 3: Final ZORUNLU kontrolü

```
if (EVP_DecryptFinal_ex(ctx, acik + n, &n) != 1) {    /* etiket BURADA doğrulanır */
    OPENSSL_cleanse(acik, sifreli_n);                /* yarı çözülmüş veriyi sil */
    return -1;
}
```

Windows CNG: BCryptDecrypt → STATUS_AUTH_TAG_MISMATCH (0xC000A002)

 En sık AEAD hatası: Final'ın dönüşüne bakmadan tampondakini kullanmak

Sabit zamanlı karşılaştırma

```
unsigned char fark = 0;
for (size_t i = 0; i < n; i++) fark |= a[i] ^ b[i];    /* her baytı dolaş */
return fark == 0;                                     /* ≈ CRYPTO_memcmp */
```

`memcmp` ilk farklı baytta durur; süre kaç baytın doğru olduğunu **sızdırır**. Hazır karşılıklar:
`CRYPTO_memcmp`, `sodium_memcmp`.

Kripto kütüphanesi kullanırken kontrol listesi

- [] Her dönüş değeri denetleniyor, hata = **dur**
- [] Nonce her seferinde yeni · etiket kullanımdan **önce** doğrulanıyor
- [] MAC karşılaştırması sabit zamanlı (`memcmp` değil)
- [] "Etiket hatası" $\neq$ "dolgu hatası" iletisi yok (**kâhin**)
- [] DES/3DES, RC4, MD5, SHA-1 (imza), ECB yok

Sahada: algoritma envanteri

- Gerçek ürün kılavuzlarında bir **algoritma envanteri** bulunur: her algoritma, hangi amaçla, hangi anahtar boyuyla.
- İyi bir eleştirel okuma malzemesi: eski envanterlerde **SHA-1** ya da **ayrı MAC'li CBC** görülür.
- Bugün yazılan bir envanterde yerini **SHA-256** ve **AEAD** almalı.

Değerlendirici nasıl test eder?

- Algoritma envanterini kaynak kodla **karşılaştırır**.
- Nonce üretimini ve tekrarını inceler.
- Şifreli veriyi bir bit değiştirerek geri verir; uygulamanın **reddettiğini** ve ayrıntılı hata vermediğini doğrular.
- MAC karşılaştırmasında `memcmp` arar.

4. IV / nonce / tuz

Üçü de gizli DEĞİL, ama...

Kavram	Nerede	Kural	Bozulursa
Tuz	Paroladan anahtar	rastgele, kayıt başına	rainbow tablo
IV	CBC	rastgele, öngörülemez	ilk blok sızar
Nonce	CTR/GCM	anahtarla asla tekrar etmesin	anahtar akışı sızar

Gizli olan tek şey **anahtardır**.

Nonce tekrarı neden felaket?

CTR/GCM: $C = P \oplus AA(\text{anahtar}, \text{nonce})$

Aynı anahtar **ve** aynı nonce iki mesajda:

$$\begin{array}{ll} C1 = P1 \oplus AA & C2 = P2 \oplus AA \\ C1 \oplus C2 = P1 \oplus P2 & \leftarrow \text{anahtar akisi yok oldu!} \end{array}$$

Saldırgan **iki düz metnin XOR'unu** ele geçirir. Birini biliyorsa diğerini çözer.

Gerçek olaylar: WEP kısa IV, Sony PS3 tekrar eden imza değeri.

Demo 2 — nonce tekrarı sızdırır

KOTU (ayni nonce):

```
C1 xor C2 = 070e1c08...5c1215
```

```
P1 xor P2 = 070e1c08...5c1215      <-- AYNI = sizdi
```

```
P2 = (C1 xor C2) xor P1 = "Toplam bakiye 45000 TL..."
```

IYI (farkli nonce):

```
C1 xor C2 != P1 xor P2              <-- sizinti YOK
```

✓ Nonce'u rastgele ya da **kalıcı sayaç** olarak üret; asla tekrar etme.

Demo 2 — kural (devamı)

- Rastgele nonce **pratiktir**; ama aynı anahtarla çok mesaj varsa (NIST önerisi: $\leq 2^{32}$) doğum günü olasılığı devreye girer.
- Alternatif: nonce'u **sayaç** yap — ama sayaç **kalıcı** saklanmalı (cihaz yeniden başlayınca sıfırlanmamalı).
- Gerçek olaylar: **WEP** kısa IV, **Sony PS3** ECDSA'da tekrar eden rastgele değer (özel anahtar çıkarıldı).

Demo 3 — ECB deseni sızdırır ("penguen")

Orijinal	AES-128-ECB	AES-256-GCM
#####	=====	: - . . - - + % * - : *
# ## ##	=##=####==##	* . # * ## : * % + ### :
# #####	=##=====##=###	: + . - . % + . # % =
#####	=====	=%+## # : ### % + %

✗ ECB: aynı blok → aynı şifreli blok, **şekil okunur kalır**

✓ GCM/CTR: her blok farklı → gürültü

ECB — kural

✓ **ECB kullanmayın.** Şifreleme için AEAD (GCM, ChaCha20-Poly1305); disk için XTS-AES.

Kod incelemesinde `ecb` geçen her yer bir **bulgudur**. Değerlendirici aynı düz metnin tekrar eden 16 baytlık bloklarını arayarak ECB'yi otomatik yakalayabilir.

Bu bölümün kuralı

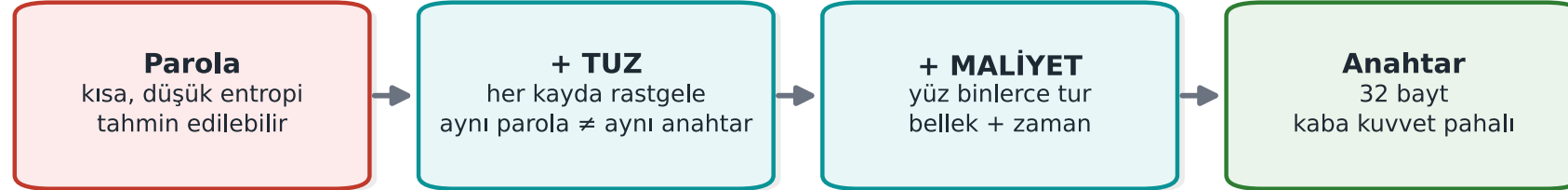
- **Tuz, IV, nonce:** hiçbirisi gizli değil, ama **her biri farklı bir tekrarsızlık** ister.
- Nonce **asla** aynı anahtarla tekrar etmesin.
- **ECB yasak.** Kip seçimi = güvenlik kararı.

5. Paroladan anahtar türetme

Paroladan anahtar türetme — şema

Paroladan anahtar türetme: iki ek, iki fayda

Argon2id / scrypt / PBKDF2



Tuz önhesaplı tabloyu, maliyet ise deneme hızını öldürür. Biri eksikse KDF işe yaramaz.

Parola neden doğrudan anahtar olamaz?

- Parola **kısa, düşük entropili ve tahmin edilebilir**.
- Doğrudan anahtar yapılırsa kaba kuvvet ucuzlaşır.
- Çözüm: paroladan anahtar türetme fonksiyonu (**KDF**).

KDF iki iş yapar

1. **Tuz:** rastgele → rainbow tablo işe yaramaz, aynı parola fark edilmez
2. **Yavaşlık:** yüzbinlerce tur → her deneme pahalı, kaba kuvvet yavaşlar

KDF	Maliyet	Ne zaman
PBKDF2	CPU (tur)	uyumluluk/FIPS
scrypt	CPU + bellek	bellek-zor
Argon2id	CPU + bellek + paralellik	birinci tercih

Demo 4 — tur sayısı, süre, tuz

- 1) TUZ: aynı parola + aynı tuz -> AYNI anahtar
 aynı parola + FARKLI tuz -> FARKLI anahtar
- 2) TUR vs SURE (WSL):
 - tur = 1000 -> 0.37 ms
 - tur = 100000 -> 39.05 ms
 - tur = 600000 -> 209.55 ms <-- OWASP 2023 asgari
 - tur = 2000000 -> 628.42 ms

Demo 4 — tuzun etkisi

- Aynı parola + **aynı** tuz → her seferinde **aynı** anahtar.
 - Tuz sabitse saldırgan bir kez tablo kurup herkesi arayabilir.
 - Aynı parola + **farklı** tuz → tamamen **farklı** anahtar.
- ➔ Tuz her kayıt için **rastgele ve benzersiz** olmalı.

Demo 4 — maliyetin etkisi

- Tur sayısı 1.000 → 2.000.000 olunca süre $\sim 0,4$ ms → ~ 628 ms'ye çıktı.
- Bu **doğrudan çarpandır**: saldırganın her denemesi de o kadar pahalılaşır.
- "Kullanıcı girişinde 200 ms" kabul edilebilir; saldırgana **milyarlarca** deneme yerine **binlerce** deneme demektir.

Demo 4 — kural

✓ Rastgele tuz + Argon2id (ya da yüksek turlu PBKDF2). ✗ Düz/tuzsuz SHA-256.

İleri: biber (pepper)

- Tuza ek olarak, veritabanında **saklanmayan**, yapılandırmada ya da HSM'de tutulan gizli bir **biber** eklenebilir.
- DB çalınırsa (biber sızmadıkça) özetler bir kat daha korunur.
- **Tuz gizli değildir; biber gizlidir.**

Bu bölümün kuralı

- Parolayı **asla** doğrudan anahtar yapma.
- Rastgele **tuz** + yeterli maliyetli bir **KDF** (Argon2id öncelikli).
- Maliyeti donanıma göre ayarla, yıllar içinde **artır**.

6. HKDF, oturum anahtarları, ileri gizlilik

Neden ana sırdan direkt kullanmıyoruz?

- Uzun ömürlü bir "ana sır" **doğrudan** kullanılmaz.
- Ondan, her amaç ve her oturum için **ayrı** anahtarlar türetilir.
- Araç: **HKDF** (RFC 5869) — iki aşamalı bir KDF.

HKDF Adım 1: Extract

- Girdi: **ana sır** + **tuz**.
- Çıktı: tekdüze bir **ara anahtar (PRK)**.
- Amaç: ham sırdaki düzensizliği tek biçim bir anahtara sıkıştırmak.

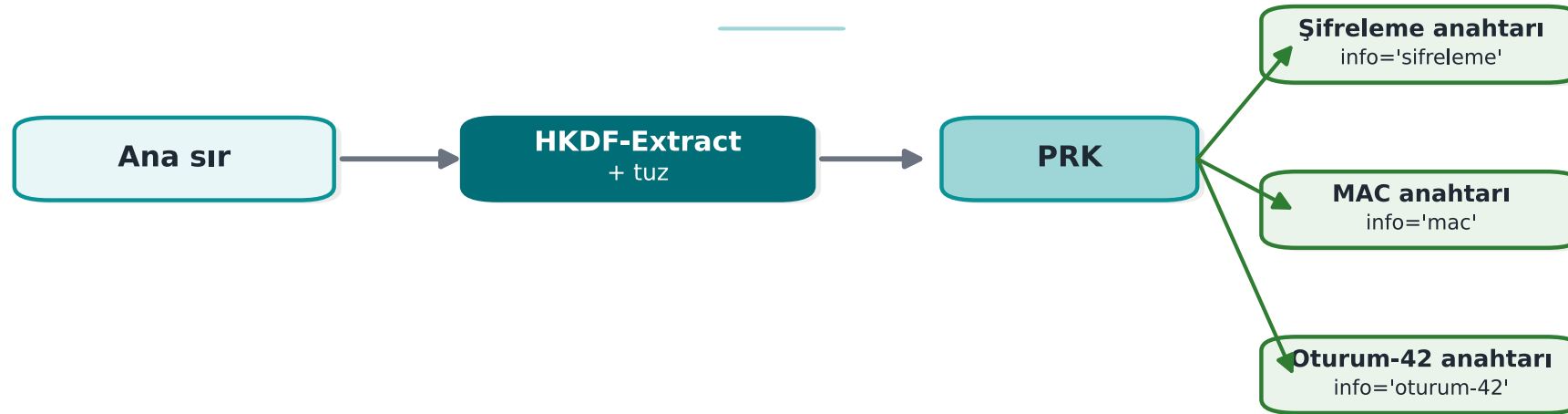
HKDF Adım 2: Expand

- Girdi: **PRK** + **info** etiketi.
- Çıktı: istenen boyda **anahtar**.
- **Farklı info** → **farklı anahtar**. Aynı PRK'dan şifreleme anahtarı, MAC anahtarı, oturum anahtarı ayrı ayrı çıkar.

HKDF: ana sırdan türev anahtarlar

HKDF: bir ana sırdan birçok anahtar

Extract (topla) → Expand (amaca göre böl)



Farklı `info` → farklı anahtar. Aynı sırdan üretilseler de biri sızarsa diğerleri güvende kalır.

Farklı `info` → farklı anahtar. Bir amacın anahtarı başka amaca kullanılmaz.

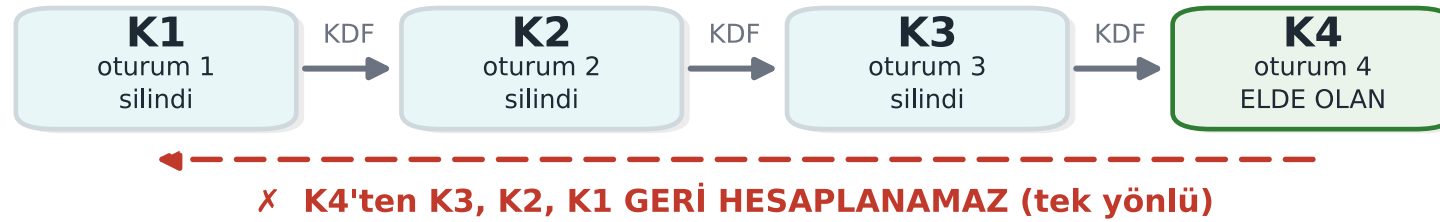
İleri gizlilik (forward secrecy) nedir?

- **İleri gizlilik:** bugünkü anahtar sızsa bile **eski oturumlar** çözülemez.
- Tek yönlü zincir: $K_0 = \text{ana sır}$, $K_i = \text{HKDF}(K_{i-1})$.
- Her adımda **eskisini sil**.

İleri gizlilik — şema

İleri gizlilik: eski anahtarlar geri hesaplanamaz

zincirin her halkası bir öncekinden tek yönlü türetilir



Bugün anahtar çalınsa bile dünkü trafik çözülemez — ileri gizliliğin tanımı budur.

Bugünkü anahtar sızsa bile **eski oturumlar** güvende. TLS 1.3'te varsayılan (geçici ECDHE ile).

Demo 5 — HKDF çıktısı

```
info='...sifreleme...' -> 2b8b4684...0e577a55  
info='...MAC...' -> 30a302d1...c55dfa84 (farkli)
```

ILERI GIZLILIK zinciri:

K1=0630f63f... K2=def84847...

K3=54b13f0f... K4=55524a8f...

^ her adimda eski anahtar silindi

Demo 5 — açıklama

- Aynı ana sırdan farklı `info` ile **farklı** anahtarlar çıktı; bir amacın anahtarı başka amaca kullanılamaz.
- İleri gizlilik zincirinde her adımda eski anahtar **silindi**.
- Elde yalnız **K4** var; zincir tek yönlü olduğu için K4'ten **K3, K2, K1 geri hesaplanamaz**.

Demo 5 — kural

- ✓ Amaca/oturuma göre ayrı anahtar · her türetmeye benzersiz `info` · kullanınca sil.

Bu bölümün kuralı

- Ana sırdan **amaca ve oturuma göre** ayrı anahtarlar türetin.
- Her türetmeye **benzersiz** bir `info` /label koyun.
- Kısa ömürlü oturum anahtarlarını **kullanınca silin**.

7. Dinamik anahtar yönetimi

Algoritma mı kırıldı, anahtar mı kötü yönetildi?

- "Algoritma kırıldı" diye haber yapılan olayların çok küçük bir kısmı gerçekten algoritmanın kırılmasıdır.
- Büyük çoğunluk **anahtar yönetimi** hatasıdır: koda gömülü anahtar, hiç değişmeyen anahtar, tek anahtarla her iş.
- Bu bölüm bu hataları önlemek için var.

Bir anahtarın hayatı (NIST SP 800-57)

Üretim → Dağıtım → Saklama → Kullanım → Yenileme → İptal → İmha

Aşama	Tipik hata
Üretim	<code>rand()</code> , sabit anahtar, istemcide üretilmemesi gereken anahtar
Dağıtım	Düz metin; yalnız TLS'e güven
Saklama	Kaynak kodda / yapılandırma dosyasında düz
Kullanım	Aynı anahtarla şifreleme ve MAC
Yenileme	Hiç değişmemesi
İmha	Yedekte, günlükte, bellekte kalan kopya

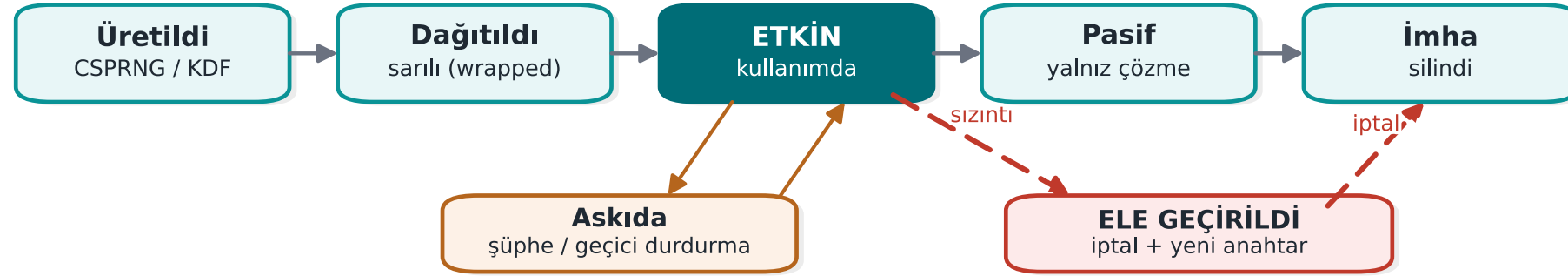
Kripto-periyot nedir?

- Bir anahtarın **kullanılmasına izin verilen süre**.
- Süre dolunca anahtar yeni veriyi **şifrelemek** için kullanılmaz.
- Eski veriyi **çözmek** için bir süre daha tutulabilir.
- Etkenler: korunan verinin miktarı, gizli kalması gereken süre, anahtarın ne kadar açıkta durduğu.

Anahtarın hayatı — şema

Bir anahtarın hayatı

üretimden imhaya — her geçiş kayıt altında



Kripto-periyodu dolan anahtar HEMEN silinmez: eski veri yeniden şifrelenene dek yalnız çözmede kalır.

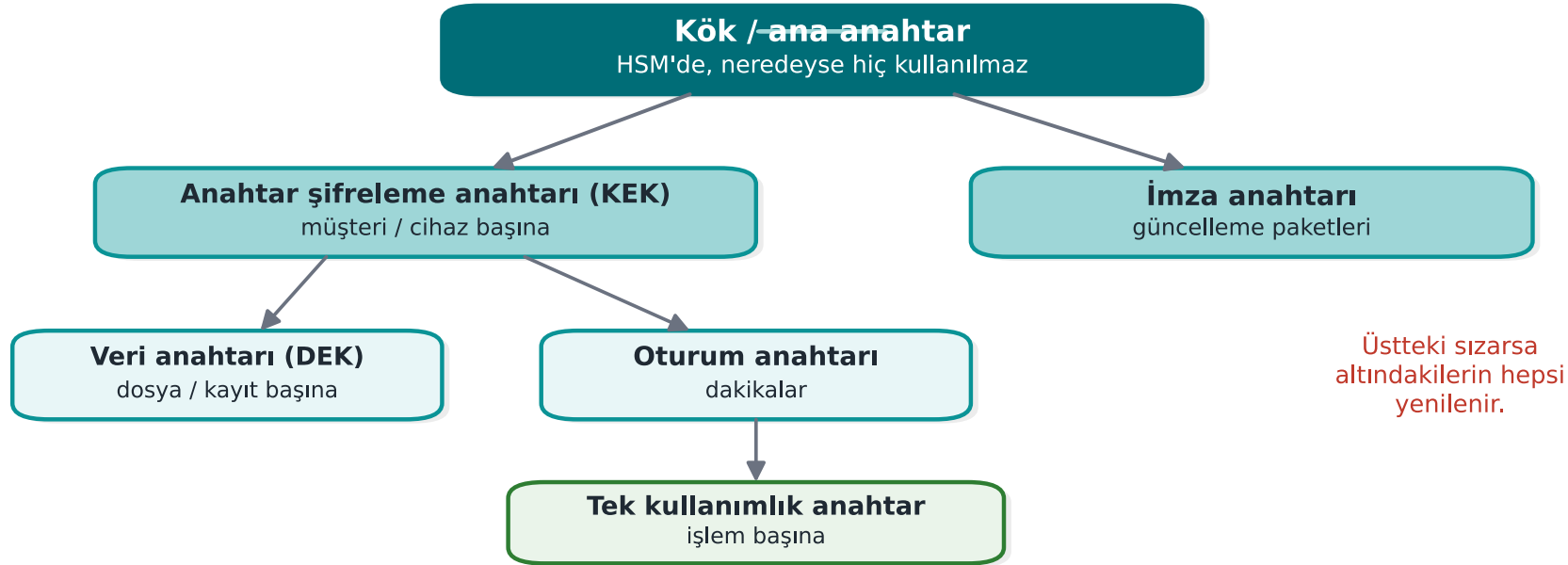
Anahtar hiyerarşisi: iki temel fikir

1. **Anahtar ayrımı:** farklı amaçlar için farklı anahtarlar (HKDF `info` ile).
2. **Zararın sınırlanması:** alt düzeydeki bir anahtarın ele geçirilmesi yalnız **o dalı** etkiler.

Anahtar hiyerarşisi

Anahtar hiyerarşisi: her işe ayrı anahtar

yukarıdakiler nadiren, aşağıdakiler sık kullanılır



- **Anahtar ayrımı:** her amaca ayrı anahtar (HKDF info)
- **Zarar sınırlama:** tek kullanımlık anahtar çalınırsa kaybedilen = **tek işlem**

Ödemede hiyerarşi ve anahtar yenileme

EMV tarzı zincir:

Kurum ana anahtarı (HSM) → **kart ana anahtarı** (kart no + sıra no) → **oturum anahtarı** (işlem sayacı) → **tek kullanımlık anahtar**

- Telefona yalnız en alttaki **sınırlı kullanım** anahtarları iner
- Tükenince sunucudan yenileri istenir (**replenishment**)
- Telefon ele geçse bile: birkaç işlemlik anahtar, sunucuda kolayca iptal

Zarflama (envelope encryption) — neden?

Büyük veriyi doğrudan ana anahtarla şifrelemek iki sorun yaratır:

- Ana anahtar **sık kullanılır** → açıkta kalma süresi artar.
- Anahtar değişince **bütün veri** yeniden şifrelenmelidir.

Zarflama — dört adım

1. Her kayıt için rastgele bir **veri anahtarı (DEK)** üretilir; veri onunla şifrelenir.
2. DEK, bir **anahtar şifreleme anahtarıyla (KEK)** sarılır.
3. KEK güvenli bir yerde durur: TPM, HSM, bulut KMS.
4. KEK yenilendiğinde yalnız küçük **DEK'ler** yeniden sarılır — veri değil.

Zarflama ve anahtar sürümü

Zarflama (envelope encryption)

veriyi DEK, DEK'i KEK şifreler



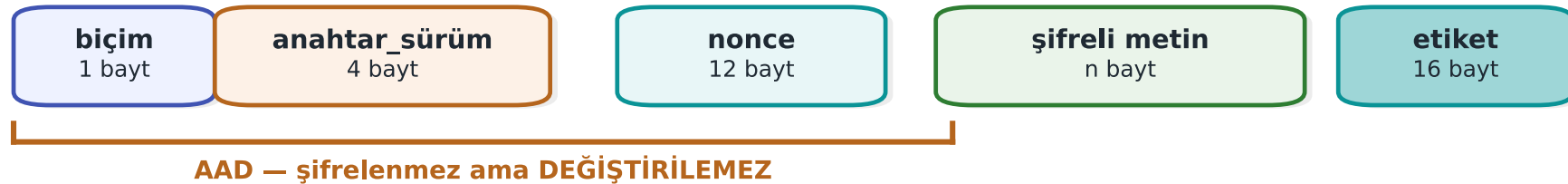
KEK yenilenince yalnız SARILMIŞ DEK'ler yeniden sarılır — terabaytlarca veri yerinde kalır.

- Okuyucu sürüme bakar → doğru anahtarı bulur
- Sürüm alanı AAD → saldırgan eski/zayıf anahtara **yönlendiremez**

Kayıt biçimi ve AAD — şema

Kayıt biçimi: anahtar yenilemeyi mümkün kılan alan

sürüm alanı olmadan anahtar değiştirilemez



Okurken önce sürüme bakılır, o sürümün anahtarıyla çözülür → eski kayıtlar bozulmadan anahtar yenilenir.

Anahtar nerede saklanır?

Seçenek	Koruma
Kaynak kod · yapılandırma · ortam değişkeni	Yok / zayıf
DPAPI (CryptProtectData)	Kullanıcı/makine hesabına bağlı
Anahtarlık · Android Keystore	OS, bazen donanım
TPM	Donanım; anahtar çıkarılamaz
HSM / bulut KMS	Anahtar HSM'den hiç çıkmaz

İstemciye yalnız **zararı sınırlı, kısa ömürlü** anahtarlar

Sahadan: şema gereksinimleri ve dört anahtar grubu

Gereksinim (Şema-A / Şema-B)	Karşılığı
Standart algoritma, güvenli yapılandırma	Doğru API kullanımı, TLS
Her anahtar: hiyerarşi, kullanım, kripto-periyot ; tek amaç	Hiyerarşi, anahtar ayrımı
Kur, güncelle, iş bitince sil	Yaşam döngüsü
Rastgele sayı çağrısı kancalanamamalı	6. hafta
Anahtarlar cihaza ve uygulamaya bağlı, farklı	Cihaz/sürüm bağlama

Gruplar: dinamik cihaz anahtarları (whitebox) · dinamik ödeme anahtarları · **tek** statik anahtar (dize gizleme) · sunucu sertifikası

Oturum anahtarı: sahadan akış

1. Sunucu 32 B oturum kimliği → yapılandırma anahtarıyla şifreli → bildirimle gelir
2. İstemci whitebox AES ile çözer
3. **Kimlik doğrulama kodu** = SHA-256(oturum kimliği parçası + cüzdan kimliği + parmak izi)
4. **Oturum anahtarı** = HMAC(yapılandırma anahtarının özeti, oturum kimliği) → 16 B AES
5. Mesaj: AES-CTR + 256 bit MAC

Oturum anahtarı: bugünkü öneri

Tasarımda	Bugün
CTR + ayrı MAC	AEAD (GCM / CCM / ChaCha20-Poly1305)
Özel HMAC türetmesi	HKDF + ayrı info
SHA-1	SHA-256 · ileri gizlilik için geçici (EC)DH

Bu bölümün kuralı

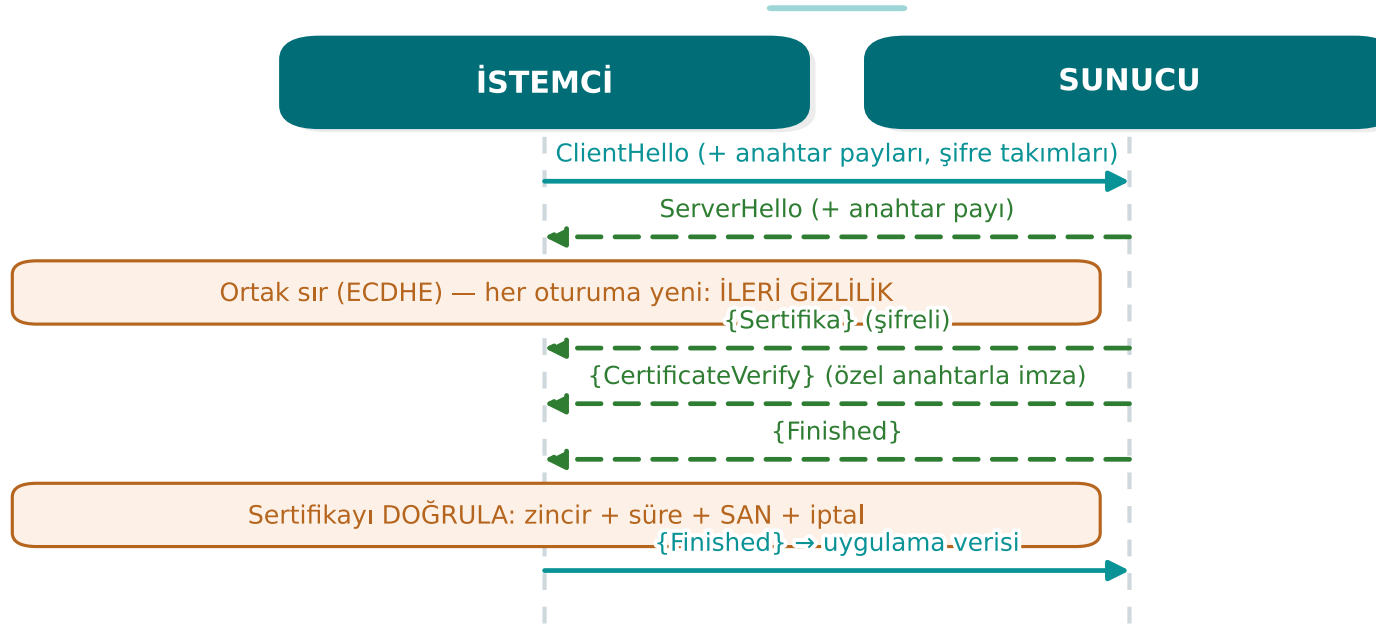
- Anahtarları bir **hiyerarşi** içinde düzenleyin; her anahtarın **tek amacı** ve bir kript-periyodu olsun.
- Veriyi **zarflayın**; anahtar sürümünü AAD ile koruyun.
- İstemci cihazına yalnız **zararı sınırlı, kısa ömürlü** anahtarlar indirin.

8. Aktarımda veri: TLS 1.3

TLS 1.3 el sıkışması

TLS 1.3 el sıkışması

tek gidiş-dönüşte anahtar anlaşması ve kimlik doğrulama



Doğrulama adımı atlanırsa şifreli ama KİMİN olduğu belirsiz bir kanal kurulur.

Tek gidiş-dönüş · her zaman geçici ECDHE → **ileri gizlilik varsayılan.**

Doğrulamanın üç katmanı

Katman	Soru
Zincir	Güvenilir CA mı imzaladı, süresi/iptali?
Ana makine adı	Sertifika gerçekten bu sunucu için mi?
Sabitleme	Beklediğim belirli anahtar mı?

⚠️ **OpenSSL varsayılanı: doğrulamaz!** `SSL_VERIFY_PEER` + `SSL_set1_host` şart.

Sabitleme (pinning) ve TOFU

- **Pinning:** sunucunun **açık anahtarının (SPKI)** SHA-256'sını göm. Zincir geçerli olsa bile anahtar tutmuyorsa **REDDET**. (Mobilde yaygın.)
- **TOFU:** PKI yoksa ilk anahtarı hatırla, değişirse uyar (SSH modeli).

TLS 1.2 → 1.3: tek RTT · yalnız güçlü AEAD takımları · zayıf özellikler kaldırıldı
· downgrade'e karşı minimum sürüm zorla.

Demo 6 — MITM → hostname + pinning

```
1 Guvensiz -> SALDIRGAN: KABUL (TEHLIKELI, MITM olurdu)
2 Dogrula  -> GERCEK   : DOGRULANDI, KABUL
3 Dogrula  -> SALDIRGAN: REDDEDILDI (kendi imzali)
4 Pin      -> GERCEK   : pin TUTTU, KABUL
5 Pin      -> SALDIRGAN: pin TUTMADI, REDDEDILDI
```

Yalnız 127.0.0.1, kendi ürettiğimiz sertifikalar. (Windows: WSL'de çalıştırın.)

"Yalnız TLS'e güvenmeme"

- TLS kanalı korur ama **uç noktalar** güvenilmez olabilir (kök alınmış telefon, hatalı pinning)
 - Yüksek güvenlikte TLS'in **içine** bir kat daha: mesaj düzeyi AEAD + MAC
 - Böylece TLS aşılsa bile gövde hâlâ şifreli → güvenlik kabuğunun en dış 2 katmanı
- ✓ Sertifika hatasını **yutma**, güven deposuna düşme (fail-closed).

TLS istemcisi: adım 1–4 (Tarif 9.1, 10.7, 10.8)

```
SSL_CTX *ctx = SSL_CTX_new(TLS_client_method());      /* 1 */
SSL_CTX_set_min_proto_version(ctx, TLS1_2_VERSION);   /* 2 sürüm düşürmeye karşı */
SSL_CTX_set_verify(ctx, SSL_VERIFY_PEER, NULL);       /* 3 DOĞRULA */
SSL_CTX_set_default_verify_paths(ctx);                /* 4 kök deposu */
```

1. Sürümden bağımsız istemci bağlamı.
2. En düşük sürümü **TLS 1.2** yapar — atlanırsa sürüm düşürme saldırısına açık.
3. Karşı tarafın sertifikasını **doğrula** — atlanırsa OpenSSL herhangi bir sertifikayı kabul eder.
4. İşletim sisteminin güvenilir kök deposunu yükler.

TLS istemcisinin adımları — şema

TLS istemcisini doğru kurmanın adımları

tek bir adım atlanırsa MITM açık kalır



En sık hata: zincir doğrulanır ama ana makine adı denetlenmez — sertifika geçerli, sunucu yanlış.

TLS istemcisi: adım 5–7

```
SSL_set_tlsext_host_name(ssl, ana_makine);           /* 5 SNI */
SSL_set1_host(ssl, ana_makine);                      /* 6 AD denetimi */
/* bağlan + el sıkış */
SSL_get_verify_result(ssl) == X509_V_OK;             /* 7 kemer + askı */
```

5. Sunucuya hangi adı istediğini söyler (SNI) — atlanırsa aynı IP'de yanlış sertifika gelebilir.
6. Sertifikadaki adın **bu** sunucu olduğunu denetler.
7. Doğrulama sonucunu açıkça sorar — bir yapılandırma hatası sessizce geçmesin (kemer + askı).

Kitap: OpenSSL'in doğrulamayan varsayılanı "**olabilecek en kötü varsayılan**"

Adım 6 neden hayati?

- Zincir doğrulaması yalnız "güvenilir CA mı imzaladı?" diye sorar
- Saldırgan **kendi** alan adı için tamamen **geçerli** bir sertifika alabilir
- Ad denetimi yoksa `banka.ornek` yerine `saldirgan.ornek` sertifikası kabul edilir

➔ 2012 "dünyanın en tehlikeli kodu" çalışması: çok sayıda uygulama ve kütüphane bu adımı atlıyordu

SPKI sabitleme

```
X509 *sert = SSL_get_peer_certificate(ssl);          /* 3.x: SSL_get1_... */
int n = i2d_X509_PUBKEY(X509_get_X509_PUBKEY(sert), NULL);
/* DER'e çevir → SHA256 → pin listesiyle CRYPTO_memcmp */
for (i = 0; i < PIN_SAYISI; i++)
    if (CRYPTO_memcmp(ozet, PINLER[i], 32) == 0) return 1;
return 0;                                           /* tutmadı: KES */
```

```
openssl s_client -connect s.ornek:443 -servername s.ornek </dev/null \
| openssl x509 -pubkey -noout | openssl pkey -pubin -outform der \
| openssl dgst -sha256 -binary | base64
```

Sabitlemeyi sürdürmek

Kural	Neden?
Sertifikayı değil SPKI 'yi sabitle	Aynı anahtarla yenilenen sertifika çalışır
Her zaman yedek pin	Birincil kaybolursa uygulama "tuğlalaşmaz"
Yedek anahtar çevrimdışı	Birlikte ele geçmesin
Yeni pin, eskisi kaldırılmadan önce dağıtılır	Güncelleme penceresi
Pin hatasını raporla	Saldırı mı, yapılandırma mı?
Bayrakla kapatılamaz	Saldırgan önce bayrağı değiştirir

Tarayıcılar HPKP'yi 2018'de bıraktı; mobil/masaüstünde sabitleme sürüyor (MASVS-NETWORK)

Windows: WinHTTP / Schannel

- WinHTTP sertifikayı ve adı **varsayılan olarak doğrular**
- Tehlike: doğrulamayı kapatan bayraklar

```
DWORD b = SECURITY_FLAG_IGNORE_UNKNOWN_CA |  
          SECURITY_FLAG_IGNORE_CERT_CN_INVALID |  
          SECURITY_FLAG_IGNORE_CERT_DATE_INVALID; /* ASLA sürümde */  
WinHttpSetOption(istek, WINHTTP_OPTION_SECURITY_FLAGS, &b, sizeof b);
```

- ✓ Test sunucusunun CA'sını geliştirme makinesinin güven deposuna ekle; kod aynı kalsın

Eleştirel okuma: sessizce açılan kapı

```
varsayilanDogrulayici.checkServerTrusted(zincir, tur);           // 1 zincir
try {
    PublicKey beklenen = depo.getCertificate("ca").getPublicKey();
    if (!Arrays.equals(beklenen.getEncoded(), zincir[0].getPublicKey().getEncoded()))
        throw new CertificateException("pin tutmadi");           // 2 pin
} catch (KeyStoreException e) {
    e.printStackTrace();                                         // 3 ??? → bağlantı KABUL
}
```

- Hata yutuldu → sabitleme **atlandı** = **fail-open**
- Ayrıca: yedek doğrulamada iptal denetimi kapalı · dosya yoksa "devam" · tek pin
-  **Fail-closed:** sonucu belirleyemiyorsan → **reddet**

En sık TLS hataları

Hata	Sonuç
Doğrulama geri çağırısı hep "geçerli" / her şeye güvenen <code>TrustManager</code>	Her sertifika kabul
Ad denetimi kapalı (<code>HostnameVerifier</code> hep <code>true</code>)	Başka sitenin sertifikası kabul
Sertifika hatasında "devam et" (web görünümü)	Araya giren fark edilmez
Hata ayıklama bayrağı sürümde	Saldırgan açar
Yedeksiz uç sertifika pini	Yenilemede uygulama çöker → pin kaldırılır
"Nasılsa TLS var"	Sunucuda/istemcide açık veri korunmaz

Kendi bağlantını test et

```
openssl s_client -connect s.ornek:443 -servername s.ornek -verify_return_error -brief  
openssl s_client -connect s.ornek:443 -tls1_1 # reddedilmeli
```

1. Kendi imzalı sertifika → **reddet** (Demo 6)
2. Başka ad için geçerli sertifika → **reddet**
3. Cihaza kendi CA'nı kur + proxy → sabitleme yapan uygulama **reddetmeli**
4. Pin hatası raporlanıyor mu?

Bu bölümün kuralı

- İstemcide doğrulamayı **elle açın**: CA deposu + `SSL_VERIFY_PEER` + `SSL_set1_host` + min. TLS 1.2.
- Yüksek riskte **SPKI sabitleme** ekleyin, her zaman **yedek pin** bulundurun.
- Hata durumunda **kapalı kalın** (fail-closed): istisnayı yutup güven deposuna düşmeyin.

9. Beklemede ve kullanımda veri

Kullanımda veri katmanları — şema

Kullanımda veri: açılma penceresini daraltan katmanlar

sır kullanılmak için bir an açılır; o an yönetilir

Kısa ömür	anahtar için hemen öncesinde aç, bitince sil (explicit_bzero)
Bellek kilidi	mlock / VirtualLock — takas alanına yazılmasını engeller
Döküm kapatma	core dump ve hata raporu devre dışı
Cihaz bağlama	anahtar cihazın kimliğine bağlanır; kopyalanırsa açılmaz
Donanım (TEE/SE)	anahtar hiç uygulamaya girmez — en güçlü katman

Hiçbiri pencereyi sıfırlamaz; amaç saldırganın o ana yetişmesini pahalı kılmaktır.

Beklemede veri: iki düzey — şema

Beklemede veri: iki düzey, iki tehdit modeli

hangi tehdide karşı koruduğunuzu önce yazın

TÜM DİSK / DOSYA

LUKS, BitLocker, XTS-AES
cihaz kapalıyken korur
açıkken her şey okunur
yedek çalınırsa yeter

ALAN / SÜTUN

yalnız hassas alan AEAD ile
uygulama açıkken bile korur
anahtar yönetimi gerekir
arama/indeks zorlaşır

Kayıp telefon → disk şifreleme yeter. Ele geçirilen sunucu → alan şifreleme şart.

Beklemede veri: dosya/alan şifreleme

- **Tüm disk/dosya:** LUKS, BitLocker, XTS-AES ya da uygulama AEAD
- **Alan/sütun:** yalnız hassas alanları uygulama katmanında AEAD ile şifrele
- **Maskleme:** **** * 4242 — şifreleme DEĞİL, maruz kalmayı azaltır (log, ekran, test, tokenizasyon)

Maskleme şifrelemenin yerine geçmez, tamamlar (farklı tehditler).

Demo 7 — SQLite'ta alanı AES-GCM ile şifrele

Saldırgan DB'yi acti:

```
1|Ayse Yilmaz|D1ED7A36... <- kart alani SIFRELI
```

Dogru anahtarla uygulama:

```
1|Ayse Yilmaz|4242-4242-4242-4242
```

Yanlis anahtar:

```
1|Ayse Yilmaz|(COZULEMEDI - etiket tutmadi)
```

ad açık, kart şifreli. Anahtar koda gömülü değil (cihaza bağlı KDF).

Linux `sqlite3`, Windows yerleşik `winsqlite3`.

Veri maskeleye: dört teknik

Teknik	Geri dönülür mü?	Kullanım
Gösterimde maskeleye	Hayır	Ekran, fiş, günlük
Tokenizasyon	Yalnız kasaya erişen	Kart saklama, ödeme
Takma adlandırma	Anahtarı olan eşleştirir	Analiz, test
Anonimleştirme	Hayır	Açık veri

Takma adlı veri hâlâ **kişisel veri** (KVKK, GDPR) · "anonim" demek çok daha güçlü bir iddia

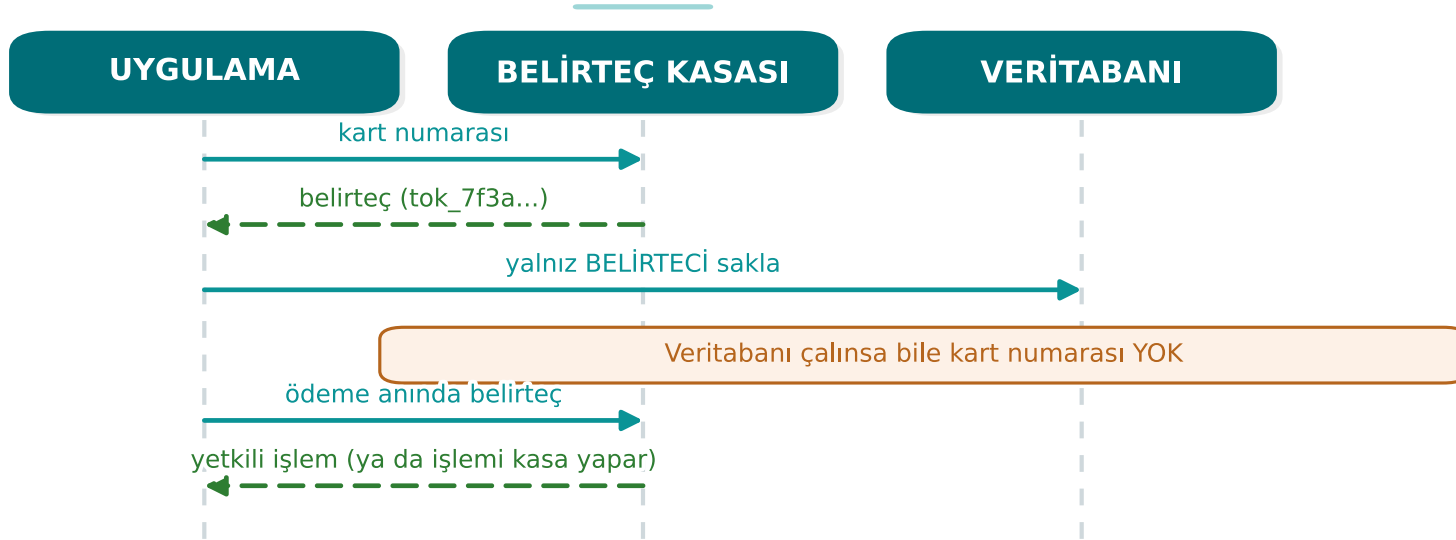
Gösterimde maskeleme

```
/* "1234567812345678" -> "*****5678" */
void pan_maskele(const char *pan, char *cikti, size_t boyut) {
    size_t n = strlen(pan, 19);
    if (boyut < n + 1) { if (boyut) cikti[0] = '\\0'; return; }
    for (size_t i = 0; i < n; i++)
        cikti[i] = (i + 4 < n) ? '*' : pan[i];
    cikti[n] = '\\0';
}
```

- PCI DSS: en fazla ilk 6 (yeni sürümlerde 8) + son 4
- Maskeleme **kaynağa yakın**: istemciye zaten maskeli gönder

Tokenizasyon: gerçek değeri kasada tut

PCI DSS denetim kapsamı kasaya daralır



Gerçek veri tek bir korunan yerde; sistemin geri kalanı anlamsız belirteç taşır.

- Veritabanı, raporlar, günlükler çalınsa bile **kart numarası yok**
- PCI DSS kapsamı küçülür = 1. haftadaki "**aktar**" yanıtı
- Mobil ödemede telefona inen = cihaza özgü **belirteç numarası**

Takma ad ve test verisi

```
kripto_hmac_sha256(takma_anahtari, 32, kimlik_no, strlen(kimlik_no), takma_ad);
```

- ❌ Düz SHA-256: 11 haneli kimlik no → hepsi denenebilir
- ✅ **Anahtarlı** HMAC; anahtar veri kümesinden ayrı · anahtar imha = fiilen anonim

Test verisi tercihi: **1. sentetik** → 2. statik maskeleye → 3. dinamik maskeleye

Günlükte maskeleye

- **Kaynakta önleme:** hassas tür, yazdırması hep maskeli
- **Çıkışta süzme:** 13–19 hane + Luhn, parola=, token= → ****

```
while ((p = strstr(p, "parola=")) != NULL) {  
    p += strlen("parola=");  
    while (*p && !isspace((unsigned char)*p)) *p++ = '*';  
}
```

⚠ Süzgeç **ikinci** hat: JSON'u tanımaz, veri zaten bellekte · sahada sürümde günlük **tamamen kaldırılır**

Kullanımda veri: 3 katman

Katman	Linux	Windows
Döküm engelle	<code>RLIMIT_CORE=0</code>	<code>SetErrorMode</code>
Belleği kilitle	<code>mlock</code>	<code>VirtualLock</code>
Güvenli sil	<code>OPENSSL_cleanse</code>	<code>SecureZeroMemory</code>

+ **kısa ömür**, kopyalama. `memset` -O2'de silinebilir; `cleanse` **silinemez**.

Demo 8 — mlock + güvenli silme

```
Katman 1 - cokme dokumu kapatildi: TAMAM  
Katman 2 - mlock ile takasa yazma engellendi: TAMAM  
Sir kullaniliyor: dolu bayt = 20  
Katman 3 - guvenli silme sonrasi dolu bayt = 0  
OPENSSL_cleanse cagrisi makine kodunda kaldı (memset gibi silinmez)
```

Cihaz bağlama: anahtarı cihaz parmak izinden **HKDF + AEAD** ile sar → başka cihazda açılmaz (Demo 9).

Bu bölümün kuralı

- Beklemede: hassas alanları **AEAD** ile şifrele, anahtarı koda gömme.
- Maskeleye şifrelemenin **yerine geçmez**, tamamlar (log, ekran, test).
- Kullanımda: sırrı **en son anda** aç, **en kısa süre** tut, kaldırılamaz biçimde **sil**.

10. Güvenlik kabukları

Demo 9 — 4 kabukla sar, ters sırada aç

```
SARMA: sir -> K1(cihaz) -> K2(depo) -> K3(oturum) -> K4(kanal)
```

```
Kabuk 4 = 128 bayt <- aktarılan paket
```

```
ACMA : K4 -> K3 -> K2 -> K1 -> sir (hepsi TAMAM, sir DOGRU)
```

```
Saldiri 1: 1 bit kurcala -> Kabuk 4 RED (etiket tutmadi)
```

```
Saldiri 2: baska cihaza kopyala -> dis 3 kabuk acilir,  
Kabuk 1 (cihaz) RED -> sir ACILAMAZ
```

✓ "Hangi tek önlem?" değil, "hangi **katmanlar**?" · bağımsız anahtarlar.

Sahadaki kabuk matrisi

Kabuk	İçerik
1	Ödeme anahtarı whitebox DES biçiminde — hiç açık değil
2	Cüzdan anahtarıyla WB-AES-CBC + cihaz + uygulama bağlama
3	Beklemede: WB-AES-CBC + SHA-256 MAC + cihaz + sürüm · Aktarımda: oturum anahtarlı AES + MAC
4	TLS + açık anahtar sabitleme + sunucu sertifikası denetimi
Dış	Kimlik doğrulama kodu (oturum anahtarı + bağlama)

Bir varlık, beş aşama

Aşama	K1	K2	K3	K4
Sunucudan aktarım	✓	✓	✓	✓
Veritabanında saklama	✓	✓	✓	—
Java tarafında çözme / API	✓	✓	—	—
Ödemede Java tarafı	✓	✓	—	—
Ödemede native taraf	✓	—	—	—

- En içte bile **bir kabuk kalır** · o anki açıklığı **RASP** korur
- "Normal" varlık: beklemede 1, aktarımda 2 kabuk → maliyet dengesi
- Saldırı algılanınca yerel veritabanı **tamamen silinir**

Bu bölümün kuralı

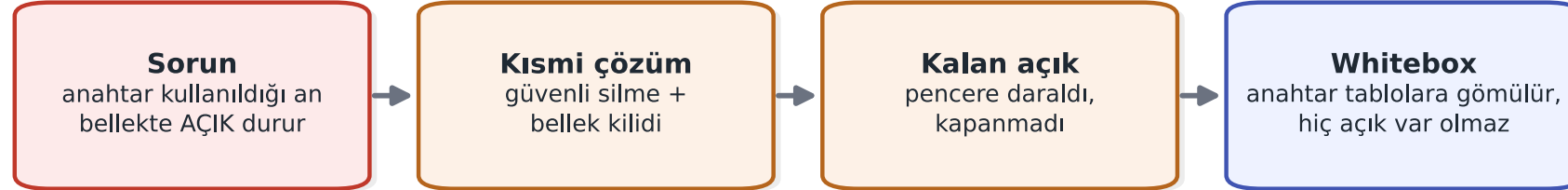
- Kritik bir varlığı korurken "hangi tek önlem?" değil, "hangi **katmanlar**?" diye sorun.
- Her aşamaya (taşıma, depolama, kullanım) bir kabuk koyun.
- Kabukları **birbirinden bağımsız anahtarlarla** kurun; birinin kırılması diğerini vermesin.

11. Whitebox kriptografiye giriş

Whitebox'a giden yol — şema

Whitebox kriptografiye neden ihtiyaç duyuldu?

11. haftanın kapısı burada açılıyor



Whitebox sihir değildir: anahtar çıkarmayı geciktirir, diğer katmanlarla anlam kazanır.

Neden whitebox?

- Bu haftanın gizli varsayımı: anahtar kullanılırken **bellekte açık durur**
- "Cihazın sahibi saldırgan" belleği okuyabilir → o anı yakalar
- Güvenli silme pencereyi **daraltır**, kapatmaz
- **Whitebox**: anahtarı tablolara gömer, ham hâli **hiç** bellekte görünmez

⚠ Sihirli değnek değil: oracle/code-lifting'e açık → diğer kabuklarla birlikte.
Ayrıntı: Hafta 11.

Yaygın yanlış anlamalar

- ✗ "Şifreledim, güvendeyim" → şifreleme yalnız gizlilik; bütünlük için AEAD
- ✗ "TLS var, MITM imkânsız" → doğrulama yapılmazsa mümkün (Demo 6)
- ✗ "SHA-256 ile parola sakladım" → tuzsuz/hızlı; Argon2id + tuz gerekir
- ✗ "Nonce gizli olmalı" → hayır, **benzersiz** olmalı; gizli olan anahtar
- ✗ "Kendi şifremi yazarım" → Kerckhoffs: güvenlik anahtarda, algoritmada değil

Proje: bu hafta (S5, S7)

- **S5 — Varlık listesi (taslak):** her hassas varlık; konum, ömür, C/I/I+, koruma
- **S7 — Veri güvenliği + kabuk matrisi:** her varlık $\times$ üç hâl $\times$ kabuklar; güvenli silme politikası

Gereksinim aileleri: CEN429-DT/DR/DU (aktarım/beklemede/kullanımda), AS, CR

S17 uyum matrisine her gereksinim için satır ekleyin.

Sınıf etkinlikleri

1. **MITM'i durdur:** Demo 6'da `SSL_set1_host` 'u kapat → hangi saldırgan kaçar?
2. **Nonce'u tekrarlat:** Demo 1'de sabit nonce → iki şifreli metni XOR'la
3. **Üç veri hâli haritası:** sağlık uygulaması için tehdit/önlem tablosu
4. **KDF maliyeti:** 100 ms veren tur sayısını bul

Kendini sinama (örnek)

1. AEAD ile AES-CTR farkı?
2. Nonce tekrarı neden felaket?
3. ECB neden kullanılmaz?
4. OpenSSL istemcisinin en tehlikeli varsayılanı?
5. Pinning neyi, hostname doğrulama neyi yakalar?
6. Güvenlik kabuğu nedir, bir örnek?

Tam liste (20 soru) ve cevaplar ders sitesinde. Quiz-1 bu tarz + kod okuma.

Kendini sinama — cevaplar (örnek 1–6)

1. **AES-CTR** yalnız gizlilik verir; **AEAD (AES-GCM)** gizlilik + **bütünlük/kimlik** doğrulamayı bir **etiketle** birlikte verir. CTR'de değişiklik fark edilmez.
2. Aynı anahtar+nonce **anahtar akışını tekrarlar**; iki şifreli metnin XOR'u açık metni sızdırır, GCM'de **kimlik anahtarı** riske girer.
3. **ECB** aynı bloğu aynı şifreli bloğa çevirir → **desenler görünür**; anlamsal güvenlik yok.
4. İstemcinin **sertifika/hostname doğrulamasını kendiliğinden yapmaması**; elle açılmazsa **MITM**.
5. **Pinning**: beklenen sunucu anahtarını (SPKI) sabitler → sahte-ama-"geçerli" sertifikayı yakalar.
Hostname doğrulama: sertifikanın doğru **alan adına** ait olduğunu yakalar.
6. **Güvenlik kabuğu**: varlığı saran koruma (C=gizlilik, I=bütünlük). Örn. anahtar dosyası = AES-GCM (C) + etiket/HMAC (I).

Kendini sinama (devam)

1. `rand()` neden güvenlik için kullanılamaz? Yerine?
2. `EVP_DecryptFinal_ex` başarısız döndü; tampondaki açık metne ne yaparsınız?
3. MAC karşılaştırmasında `memcmp` neden sakıncalı?
4. Kripto-periyodu dolan anahtar hemen silinir mi?
5. Zarflamada KEK yenilenince neden veri yeniden şifrelenmez?

Kendini sinama — cevaplar (1–5)

1. `rand()` **istatistiksel**dir, tahmin edilebilir → CSPRNG kullanın (`RAND_bytes` / `getrandom`).
2. Çözme başarısızsa tampondaki açık metni **güvenli silin** (`memset_s`) ve **reddedin** — bütünlük doğrulanmadı, kullanmayın.
3. `memcmp` **erken çıkar** → **zamanlama sızıntısı**; `CRYPTO_memcmp` / sabit zamanlı karşılaştırma kullanın.
4. **Hayır**. Yeni şifrelemede kullanılmaz ama eski veri yeniden şifrelenene dek **yalnız çözmede** tutulur.
5. Veri **DEK** ile şifrelidir; KEK yenilenince yalnızca **DEK'ler yeniden sarılır** (rewrap), veri değil.

Kendini sinama (devam)

6. `SSL_set1_host` çağrılmazsa ne olur?
7. Sabitlemede neden SPKI ve neden yedek pin?
8. "Fail-open" nedir? Bir örnek
9. Kimlik numarasının düz SHA-256'sı neden takma ad olamaz?
10. Tokenizasyon PCI DSS kapsamını neden küçültür?

Kendini sinama — cevaplar (6–10)

6. `SSL_set1_host` yoksa **hostname doğrulanmaz**; başka ada ait **geçerli** sertifika kabul edilir → MITM.
7. **SPKI** sertifika yenilense de (anahtar aynıysa) **değişmez**; **yedek pin** anahtar değişiminde uygulamanın **"tuğlalaşmasını"** önler.
8. **Fail-open**: hata durumunda "geçti/izin ver" sayılması; örn. **yutulan** `KeyStoreException` ile doğrulamanın atlanması.
9. Kimlik no'nun değer uzayı **küçük/tahmin edilebilir** → düz SHA-256 **kaba kuvvetle** geri döner; **anahtarlı HMAC** gerekir.
10. **Tokenizasyon**: gerçek PAN yalnız **kasada** kalır; sistemin kalanı token tutar → **PCI DSS denetim kapsamı** daralır.

Gelecek hafta

Hafta 4 — Kod Sağlamlılaştırma: C/C++

Güvenli bellek yönetimi · sanitizier'lar · derleyici/OS korumaları
(stack canary, ASLR, DEP/NX, CFI) · kontrol akışı düzleştirme

Kaynak: Viega & Messier, Tarif 3.1–3.5, 13.1

Ek · Bir veriyi baştan sona korumak

Senaryo

Bir uygulama, kullanıcının bir "notunu" korumalı:

- sunucudan gelir (aktarımda)
- diskte durur (beklemede)
- ekranda gösterilir (kullanımda)

Üç hâli de koruyalım.

Aktarımda · adım

- TLS 1.3 ile taşı.
- Sertifika zinciri + SAN denetimi.
- Varsa SPKI pin + yedek pin.

Beklemede · adım

- **AES-256-GCM** ile şifrele (AEAD).
- Nonce benzersiz.
- Anahtar TEE/HSM ya da paroladan KDF.

Kullanımda · adım

- En kısa süre bellekte tut.
- İş bitince **sil** (`memset` benzeri).
- RASP + kısa ömür.

Anahtar nereden?

- Ana parola → **PBKDF2/Argon2** (tuzlu, yavaş) → ana anahtar.
- Ana anahtar → **HKDF** → veri anahtarı, oturum anahtarı.
- Her anahtarın **ayrı** görevi.

Bütünlük

- AEAD zaten bütünlük veriyor.
- Aktarımda TLS, beklemede GCM etiketi.
- Kurcalama → çözme **reddedilir**.

Senaryo · özet

Hâl	Koruma
Aktarımda	TLS 1.3 + zincir
Beklemede	AES-GCM + KDF
Kullanımda	kısa ömür + silme

Ek · Klasik kriptoloji hataları

Hata · nonce tekrarı

- GCM'de aynı nonce iki kez → **felaket** (anahtar/veri sızabilir).
- Sayaç ya da yeterli rastgele nonce.

Hata · zayıf rastgelelik

- `rand()` ile anahtar/nonce → tahmin edilebilir.
- **CSPRNG** kullan (`getrandom` , `BCryptGenRandom`).

Hata · parolayı doğrudan anahtar yapmak

- Parola düşük entropili; doğrudan anahtar **zayıf**.
- **KDF** ile türet (tuzlu, yavaş).

Hata · ECB kipi

- Desen sızar ("penguen").
- AEAD kullan.

Hata · şifreleme ama bütünlük yok

- Yalnız gizlilik yetmez; kurcalanabilir.
- AEAD ya da encrypt-then-MAC.

Hata · sabit zamanlı olmayan karşılaştırma

- MAC/parola karşılaştırması erken çıkarsa **zamanlama** sızdırır.
- Sabit zamanlı karşılaştır.

Ek · Çözümlü kendini sinama

Soru 1

Verinin üç hâli ve her birinin korunması?

Cevap: Aktarımda (TLS), beklemede (AEAD şifreleme), kullanımda (kısa ömür + silme).

Soru 2

GCM'de nonce tekrarı neden felaket?

Cevap: Aynı anahtar+nonce altında gizlilik ve bütünlük çöker; anahtar akışı ve veri sızabilir. Nonce benzersiz olmalı.

Soru 3

Parola neden doğrudan anahtar olamaz?

Cevap: Düşük entropili ve tahmin edilebilir. KDF (PBKDF2/Argon2) ile tuzlu, yavaş türetilir.

Soru 4

AEAD ne sağlar, hangi ayrı adımların yerine geçer?

Cevap: Gizlilik + bütünlüğü birlikte; ayrı şifreleme + MAC adımlarının yerine.

Soru 5

İleri gizlilik ne demek?

Cevap: Her oturum yeni anahtar; uzun vadeli anahtar sızsa bile eski oturumlar çözülemez.

Soru 6

IV/nonce/tuz gizli mi? Kuralı ne?

Cevap: Gizli değildir ama **benzersiz/tekrarsız** olmalı; nonce tekrarı ve sabit tuz tehlikelidir.

Soru 7

CSPRNG neden gerekli?

Cevap: `rand()` tahmin edilebilir; anahtar/nonce/tuz için kriptografik güvenli üreteç (OS) gerekir.

Soru 8

Anahtar hiyerarşisi neden var?

Cevap: Tek anahtar her işte kullanılmaz; ana anahtardan türev anahtarlar üretilir, her birinin ayrı görevi ve ömrü olur.

Sözlük

Terim	Anlam
Üç hâl	Aktarımda/beklemede/kullanımda
AEAD	Şifreleme + bütünlük
CSPRNG	Güvenli rastgele üreteç
KDF/HKDF	Anahtar türetme
Nonce/tuz	Benzersiz/rastgele yardımcı değer
İleri gizlilik	Eski oturumlar sonra çözülemez

Son söz (3. hafta)

Veriyi **üç hâlinde de** koru; doğru kip (AEAD), güvenli rastgelelik, KDF ile anahtar, benzersiz nonce ve iyi yönetilen anahtar hiyerarşisi.

10. haftada PKI ve sertifikalarla derinleşeceğiz.